

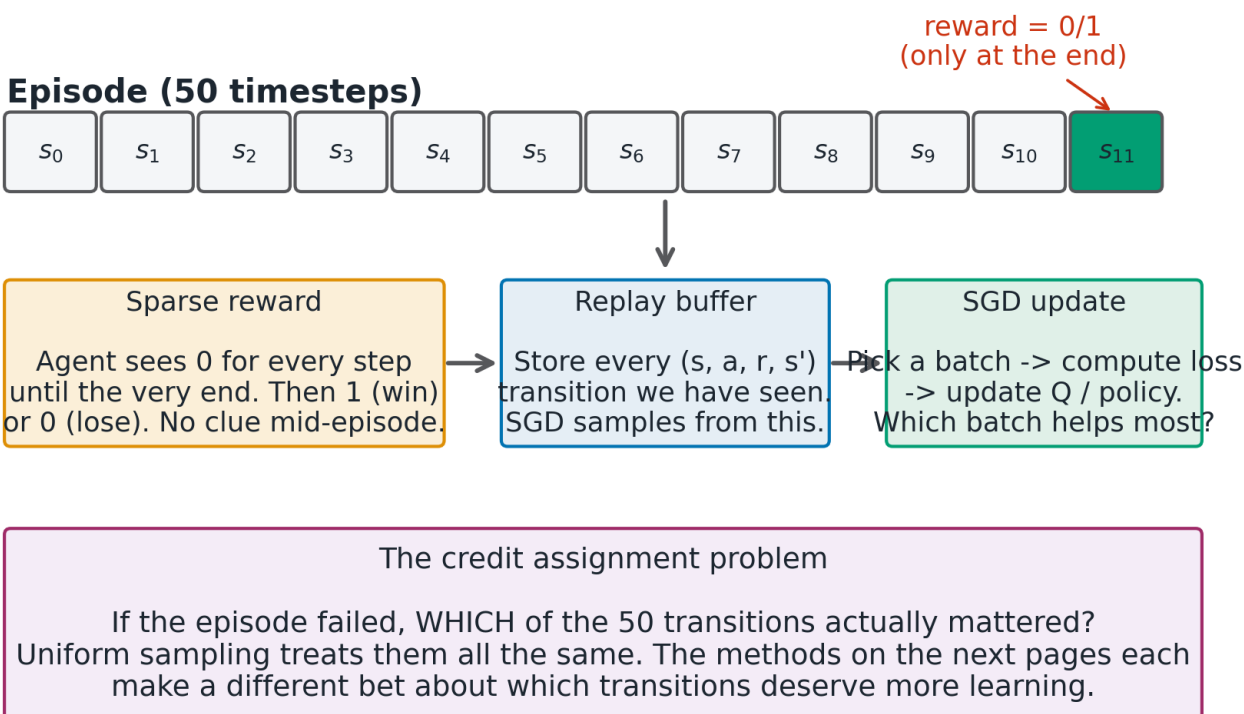
Replay-Buffer Methods, a Visual Explainer

From uniform replay to verified counterfactuals — the family tree behind RL_project_185. · Daniel Grant, CS 285, May 2026.

The problem we are trying to solve

A robot tries to do a task. It mostly fails. We want it to learn from those failures. To do that, every step of every attempt gets stuffed into a big bucket called the **replay buffer**, and stochastic gradient descent pulls samples out and uses them to improve the policy.

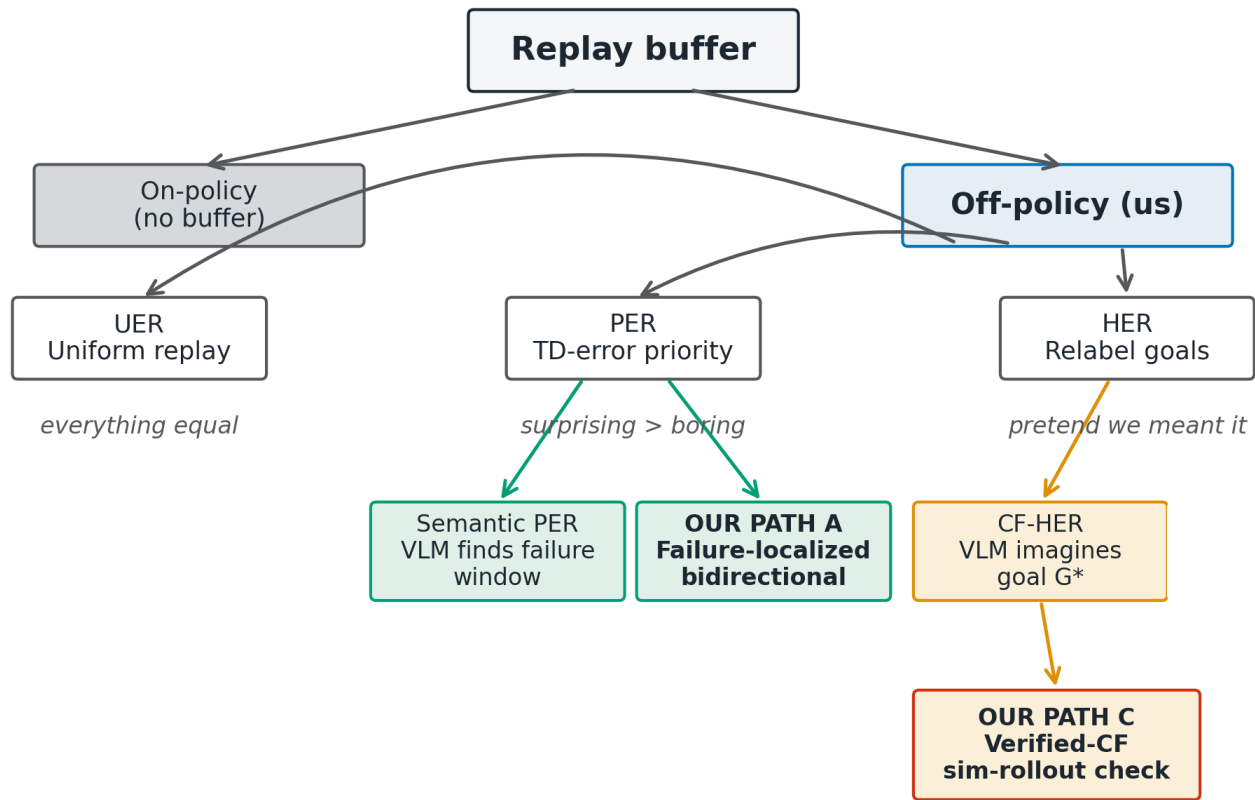
Two things make this hard. First, the reward is **sparse**: the robot sees a 0 for almost every transition and only finds out whether it succeeded at the very last step. Second, even when the episode *does* finish with a 1, we have no idea **which** of the 50 transitions in that episode actually deserves credit. This is the **credit assignment problem**, and it is what every method on the next pages is attacking.



Every episode produces a strip of transitions (s, a, r, s'). Only the last one carries a non-zero reward. The buffer collects them all; SGD has to figure out which ones are worth learning from.

The family tree

Six methods, all answering the same question ("which transitions should we sample, and with what reward?") in increasingly clever ways. The branches in green are this project's contributions.



Each leaf method is a different bet about how to fix the credit-assignment problem. UER bets on "all transitions matter the same." PER bets on "surprising transitions matter more." HER bets on "if we missed the goal, pretend we meant to land where we did." The leaves below them refine these bets using a vision-language model (VLM).

One-line intuition for each

- **UER** (Uniform Experience Replay): sample any transition with equal probability. Boring but unbiased — the baseline.
- **PER** (Prioritized Experience Replay, Schaul 2016): sample transitions where the agent is most *surprised* (high TD-error) more often. "Learn from your mistakes."
- **HER** (Hindsight Experience Replay, Andrychowicz 2017): when the agent fails, relabel the goal to be wherever the agent *actually* ended up. Now every episode is a success at *something*.
- **Semantic PER** (this project, Path A): ask a vision-language model where the failure happened, then up-weight the transitions around that timestep on top of PER.
- **Counterfactual HER** (CF-HER): ask the VLM "if you were going to succeed, what should the goal have been?" Use the VLM's answer as the relabel goal.
- **Verified-CF** (this project, Path C, agent N1): same as CF-HER, but only accept the VLM's proposal if it actually works when replayed in the simulator. "Trust but verify."

Side-by-side comparison

All six methods in one table. Read the rows top-to-bottom — each row builds on the previous one.

Method	What it does	When it helps	When it hurts	Key knob	Published example
UER	Sample transitions uniformly at random.	Always safe baseline; cheap; unbiased.	Wastes capacity on transitions the agent already understands.	(none)	Mnih et al. 2015 (DQN on Atari)
PER	Sample with probability proportional to $ \text{TD-error} ^\alpha$.	Dense rewards or any setting where TD-error is informative.	Sparse-reward tasks: TD-error is ~ 0 everywhere until the goal.	α (priority sharpness), β (IS correction)	Schaul et al. 2016 (Prioritized Replay, DQN Atari)
HER	On failed episodes, relabel goal = achieved state; recompute reward.	Goal-conditioned manipulation with sparse rewards.	Tasks without a clear achieved-goal representation; can confuse the policy if the relabel goal is unreachable from earlier states.	k (number of relabels per episode), strategy ("future", "final")	Andrychowicz et al. 2017 (Fetch tasks)
Semantic PER	Multiply PER priority by a VLM-derived failure-window weight.	Long-horizon tasks where one transition is causally responsible for failure.	Tasks where the VLM cannot identify the failure frame; flat trajectories.	W (window radius), w_max (boost magnitude)	This project (Fetch, 1M steps); Sharony et al. 2026 (MiniGrid, OGBench)
CF-HER	VLM proposes a counterfactual goal G^* ; relabel and recompute reward.	Cases where the policy was "close to right" and the VLM can verbalize the near-miss.	If the trajectory never reached G^* , reward is still 0 -- no winning transition is created. Mechanistic weakness.	p_counterfactual (frequency of CF relabels)	This project, Path C v1 (synthetic + real Fetch failures)
Verified-CF	VLM proposes a corrective action; sim is rolled out; accept only if env reward fires.	Whenever the simulator is cheap to fork; turns the VLM into a generator of provably-valid transitions.	Needs a forkable simulator. VLM still has to produce a coherent action sequence.	rollout horizon, accept threshold	This project, Path C v2 (agent N1, 4/4 smoke pass)

How to read this table: each method fixes a specific weakness of the previous one. UER -> PER fixes "every transition treated equally." PER -> HER fixes "TD-error is zero in sparse-reward land." HER -> Semantic PER fixes "HER only relabels the achieved goal, it does not tell you which timestep mattered." Semantic PER -> CF-HER changes the question from "which transition?" to "which goal?" CF-HER -> Verified-CF fixes "the VLM's imagined goal might be a lie."

Pseudocode — the easy three

These three you can implement in an afternoon. Read them as templates: the later methods (Semantic PER, CF-HER, Verified-CF) are *extensions* of these three, not replacements.

UER — Uniform Experience Replay

Pull a random transition with equal probability. That is the whole algorithm.

```
i = uniform_random(0, len(buffer))
transition = buffer[i]
```

PER — Prioritized Experience Replay

Each transition i has a priority $p_i = |\text{TD-error}_i|^\alpha$. Pull it proportional to its priority. Re-weight the gradient with an importance-sampling correction so that the loss is still unbiased.

```
p_i = (|TD_i| + eps) ** alpha
i ~ Categorical(p_i / sum_j p_j)
w_i = ((N * p_i / sum_j p_j) ** -beta) / max_w # IS correction
loss = w_i * (Q(s_i, a_i) - target_i) ** 2
```

HER — Hindsight Experience Replay

On a failed episode τ , pick a future state, relabel the goal to be that state's `achieved_goal`, recompute the reward function on the relabeled trajectory, and push the relabeled transitions back into the buffer.

```
for tau in failed_episodes:
    for t in range(len(tau)):
        # pick a future timestep t' > t
        t_prime = random_future(t, len(tau))
        new_goal = tau[t_prime].achieved_goal
        new_reward = env.compute_reward(tau[t].achieved_goal, new_goal)
        buffer.push(tau[t] with desired_goal=new_goal, reward=new_reward)
```

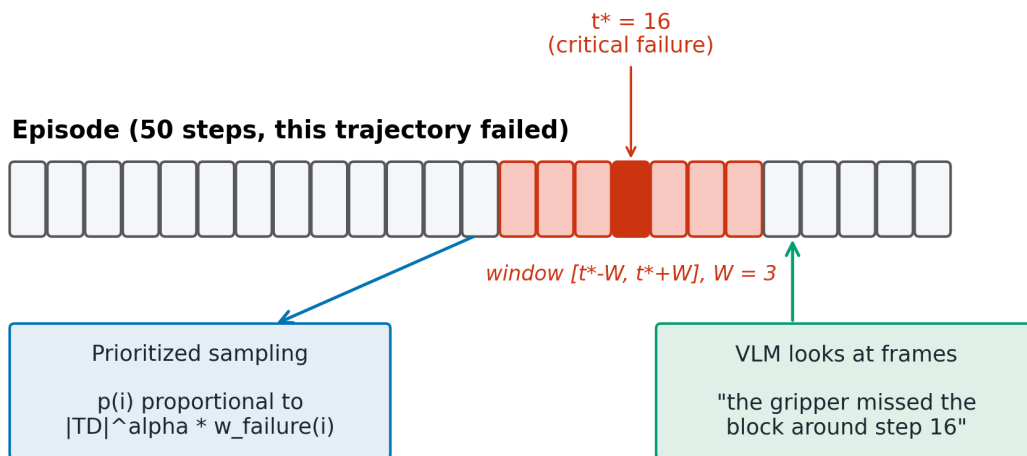
Mental model: UER is the bucket. PER is a smarter way to pull from the bucket. HER is a way to *add more useful things to the bucket*. PER and HER are orthogonal — you can do both. (Most modern manipulation papers do.)

The novel three, with diagrams

All three add a vision-language model (VLM) to the loop. They differ in *what they ask the VLM to produce*, and how much they trust the answer.

Semantic PER — "which timestep was the mistake?"

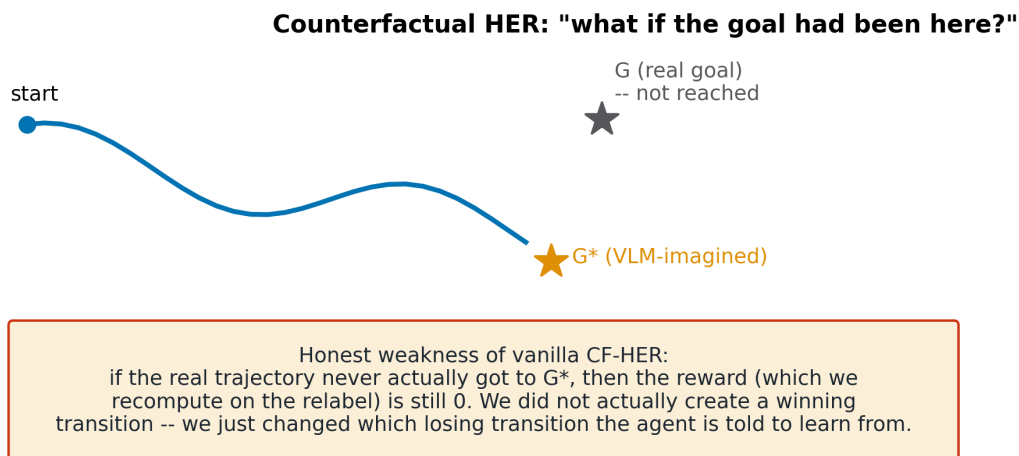
Show the VLM keyframes from the episode and ask it to point to the **critical failure timestep t^*** . Build a soft window of radius W around t^* and multiply each transition's PER priority by that window. Transitions near the mistake get sampled much more often.



The VLM picks the critical moment; we boost a small window around it. Outside the window, behaviour is identical to vanilla PER.

Counterfactual HER — "what should the goal have been?"

Instead of asking about timesteps, ask the VLM about **goals**. On a failed episode the VLM proposes an imagined goal G^* ; we relabel `desired_goal =  $G^*$` and recompute the reward, exactly like HER.

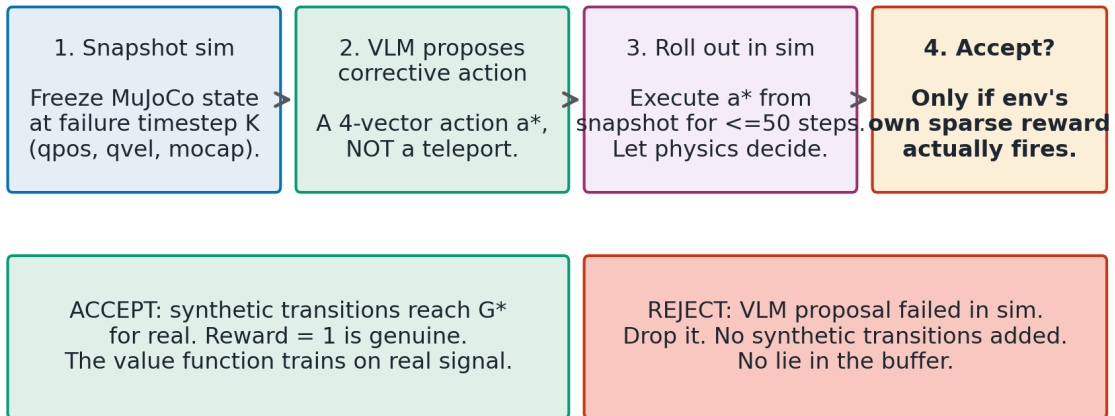


CF-HER swaps real goal G for VLM-imagined G^* . Honest weakness: if the trajectory never reached G^* , the recomputed reward is still 0.

Verified-CF — "prove your counterfactual works"

The fix. Ask the VLM for a **corrective action sequence**, snapshot the simulator at the failure timestep, roll out the VLM's action in a forked sim, and accept only if the environment's own reward fires. Accepted transitions are guaranteed physically valid with a real reward of 1.

Verified-CF (our Path C, agent N1)



Key property: teleport is structurally impossible -- you cannot teleport a block with a 4D action.

Verified-CF gates VLM proposals through the simulator. Teleport pathologies are eliminated by construction.

Why each method beats the previous (and when)

One short paragraph per method: the design pressure, what it fixed, and what is still wrong.

UER → PER

Why: uniform sampling wastes compute on transitions the agent already understands. **Fix:** PER concentrates sampling on high TD-error transitions. **Still wrong:** in sparse-reward tasks, TD-error is ~0 almost everywhere, so PER degrades back toward uniform.

PER → HER

Why: PER cannot extract signal from rewards that are never observed. **Fix:** HER manufactures successful transitions by relabeling the goal of failed episodes — every failure becomes a success at *some* goal. **Still wrong:** HER picks a goal but not a timestep, and assumes the env exposes `achieved_goal`.

HER → Semantic PER

Why: in long-horizon manipulation, one or two transitions out of 50 typically cause the failure (grasp slipped, gripper missed by 2 cm). **Fix:** a VLM identifies where the failure happened; we boost those transitions on top of PER. **Still wrong:** VLMs are noisy — our 36-run ablation shows GPT-4o under-performs a privileged-state heuristic.

Semantic PER → CF-HER

Why: Semantic PER reweights existing transitions but does not change *what they say*. They still carry the failed reward. **Fix:** CF-HER asks the VLM to imagine an alternate goal G^* and relabels accordingly. **Still wrong:** nothing forces G^* to have been reached. On Push the VLM frequently proposes $G^* = \text{desired_goal}$ (teleport); reward stays 0.

CF-HER → Verified-CF

Why: a synthetic transition that lies about its reward poisons the value function. **Fix:** Verified-CF executes the VLM's proposal in a forked simulator and accepts only if the env reward fires. **Still wrong:** needs a forkable sim. The VLM still must produce a coherent action sequence; if not, fall back to plain HER.

Empirical findings from our project (Fetch, 250k–1M steps)

- **Path A:** Semantic PER with heuristic Oracle beats vanilla PER (**0.417 vs 0.383**, mean over 3 seeds, 1M steps).
- **Path A (GPT-4o):** swap the Oracle for a VLM and it drops below PER (**0.31 vs 0.38**). Mechanism works; the VLM is the bottleneck.
- **Path C v1 (CF-HER):** does not beat plain HER on Fetch at the 250k-step horizon. Teleport-collapse on Push dominates.
- **Path C v2 (Verified-CF):** 4/4 smoke pass on sim-grounded verification; full training run launching today.

Headline lesson: adding a VLM is not automatic. It helps when the VLM has a clear job, is calibrated against ground truth, and its outputs are structurally constrained (window weights, not arbitrary coordinates). It hurts when you ask it to invent physical-world facts without verifying them. Verified-CF is the principled response.